

Rasterization & The Graphics Pipeline

Fast Approximations for Real-Time Graphics

Ashrafur Rahman

Adjunct Lecturer

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology (BUET)

Index

Introduction

- Philosophy

- The Speed vs Accuracy Trade-off

- Realtime Challenge

The GPU Evolution

- History

- GPU Domination

- Modern GPU Architecture

The Modern Graphics Pipeline

- Definition

- Stages

- Advanced Pipeline

- Data Flow Through the Pipeline

- Pipeline Stage Types

APIs & Shading Languages

- Graphics APIs

- Shading Languages

- GLSL Overview

Input Assembly

- Overview

Introduction

Why Rasterization?

Why Rasterization?



Valorant - 120 FPS Gaming

Why Rasterization?



Valorant - 120 FPS Gaming



Up - 30 hours per frame

- **Real-time constraint:** Games need 60-120 FPS

Why Rasterization?



Valorant - 120 FPS Gaming



Up - 30 hours per frame

- **Real-time constraint:** Games need 60-120 FPS
- **Interactive experience:** User input must feel responsive

Why Rasterization?



Valorant - 120 FPS Gaming



Up - 30 hours per frame

- **Real-time constraint:** Games need 60-120 FPS
- **Interactive experience:** User input must feel responsive
- **Trade-off:** Sacrifice physical accuracy for speed

Why Rasterization?



Valorant - 120 FPS Gaming

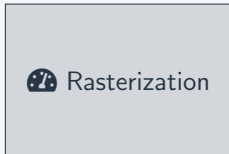


Up - 30 hours per frame

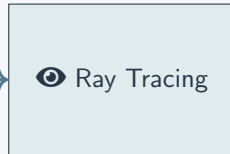
- **Real-time constraint:** Games need 60-120 FPS
- **Interactive experience:** User input must feel responsive
- **Trade-off:** Sacrifice physical accuracy for speed
- **Goal:** Images that look good enough, delivered fast enough

Rasterization vs Ray Tracing: The Fundamental Choice

Fast Approximations



Physical Accuracy



Trade-off

Rasterization:

- 60-240 FPS
- Clever approximations
- Hardware optimized
- "Good enough" quality

Ray Tracing:

- ≈ 0 FPS
- Physical simulation
- Computationally heavy
- Photorealistic

The Real-Time Graphics Challenge

Time Budget at 60 FPS

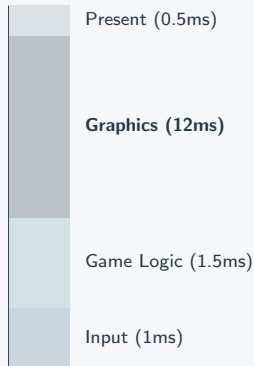
$$\frac{1}{60} = 16.67 \text{ milliseconds per frame}$$

What needs to happen:

- Process input
- Update game logic
- Render graphics
- Present to screen

Graphics budget: ~10-12ms

16.67ms

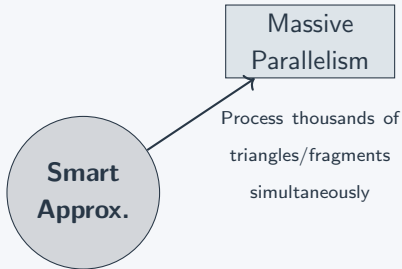


How Rasterization Achieves Speed

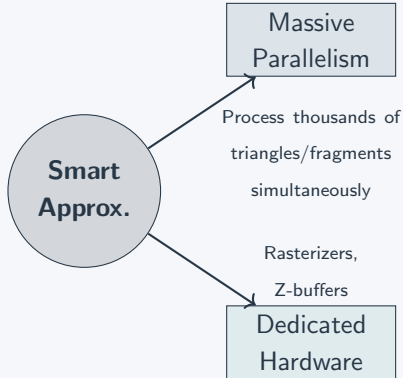


**Smart
Approx.**

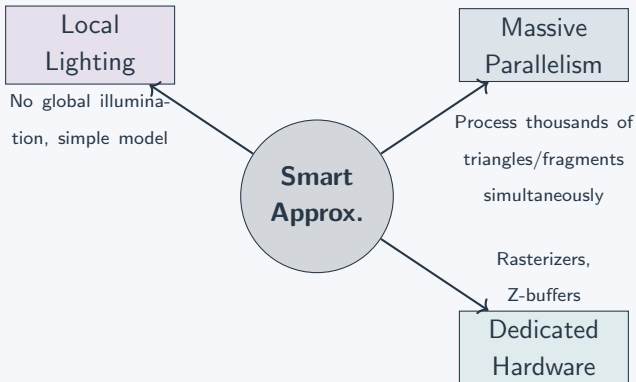
How Rasterization Achieves Speed



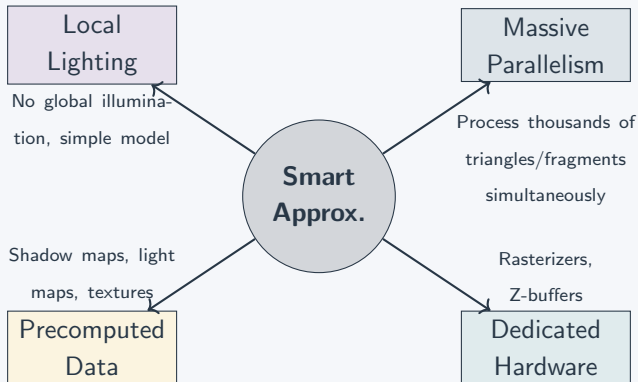
How Rasterization Achieves Speed



How Rasterization Achieves Speed



How Rasterization Achieves Speed



The Clever Approximations

What We Skip

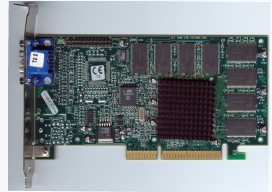
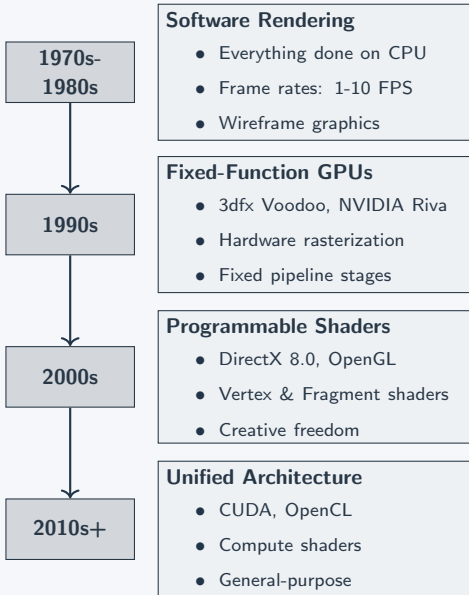
- **Global illumination:**
No light bouncing
- **Perfect shadows:** Use shadow maps
- **Perfect reflections:**
Use environment maps
- **Complex materials:**
Simplified BRDFs

What We Gain

- **Predictable performance:** Linear with triangle count
- **Hardware optimization:**
Purpose-built silicon
- **Real-time interaction:**
Immediate feedback
- **Scalable quality:**
Adjust for performance

The GPU Evolution

A Brief History



3dfx Voodoo 3 - 1999



NVIDIA GeForce 5090 - 2025

Why GPUs Dominate Graphics

CPU

4-16 complex cores

Large caches

Branch prediction

Out-of-order execution

Why GPUs Dominate Graphics

CPU

- 4-16 complex cores
- Large caches
- Branch prediction
- Out-of-order execution

GPU

- 1000s of simple cores
- Small caches
- SIMD execution
- Throughput optimized

Why GPUs Dominate Graphics

CPU

- 4-16 complex cores
- Large caches
- Branch prediction
- Out-of-order execution

GPU

- 1000s of simple cores
- Small caches
- SIMD execution
- Throughput optimized

Serial Tasks

- Complex logic
- Branching
- Low latency

Why GPUs Dominate Graphics

CPU

- 4-16 complex cores
- Large caches
- Branch prediction
- Out-of-order execution

Serial Tasks

- Complex logic
- Branching
- Low latency

GPU

- 1000s of simple cores
- Small caches
- SIMD execution
- Throughput optimized

Parallel Tasks

- Simple operations
- Same instruction
- High throughput

Why GPUs Dominate Graphics

CPU

- 4-16 complex cores
- Large caches
- Branch prediction
- Out-of-order execution

Serial Tasks

- Complex logic
- Branching
- Low latency

GPU

- 1000s of simple cores
- Small caches
- SIMD execution
- Throughput optimized

Parallel Tasks

- Simple operations
- Same instruction
- High throughput

Perfect Match: Graphics + GPU

Graphics pipeline stages process thousands of vertices/fragments *independently*

⇒ Ideal for massively parallel GPU architecture

Modern GPU: The Graphics Powerhouse

Hardware Implementation

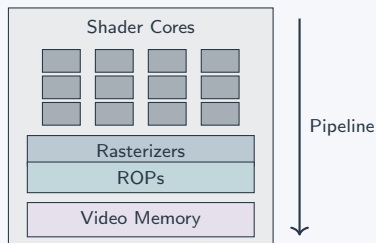
GPU handles entire pipeline:

- **Vertex processing:** Shader cores
- **Rasterization:** Fixed-function units
- **Fragment processing:** Shader cores
- **Memory operations:** ROPs

GPU Driver handles:

- Command submission
- State management
- Resource allocation

GPU Chip



The Modern Graphics Pipeline

Why a Graphics Pipeline?

The Rasterization Process

- GPU process vast numbers of **vertices** and **pixels** every frame (millions per second)

Why a Graphics Pipeline?

The Rasterization Process

- GPU process vast numbers of **vertices** and **pixels** every frame (millions per second)
- Each undergoes a **series of identical operations**

Why a Graphics Pipeline?

The Rasterization Process

- GPU process vast numbers of **vertices** and **pixels** every frame (millions per second)
- Each undergoes a **series of identical operations**
- The operations can be divided into **stages**

Why a Graphics Pipeline?

The Rasterization Process

- GPU process vast numbers of **vertices** and **pixels** every frame (millions per second)
- Each undergoes a **series of identical operations**
- The operations can be divided into **stages**
- Each stage can be **efficiently** implemented in hardware

Why a Graphics Pipeline?

The Rasterization Process

- GPU process vast numbers of **vertices** and **pixels** every frame (millions per second)
- Each undergoes **a series of identical operations**
- The operations can be divided into **stages**
- Each stage can be **efficiently** implemented in hardware
- Stages can be **parallelized** for high throughput

Why a Graphics Pipeline?

The Rasterization Process

- GPU process vast numbers of **vertices** and **pixels** every frame (millions per second)
- Each undergoes a **series of identical operations**
- The operations can be divided into **stages**
- Each stage can be **efficiently** implemented in hardware
- Stages can be **parallelized** for high throughput

The Graphics Pipeline

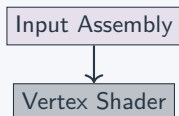
The graphics pipeline is a sequence of stages that process vertices and fragments in parallel, transforming 3D models into 2D images.

Pipeline Stages at a Glance

Input Assembly

Input Assembly: Pull vertex data (positions, normals, UVs) into the pipeline.

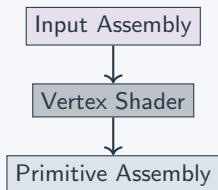
Pipeline Stages at a Glance



Input Assembly: Pull vertex data (positions, normals, UVs) into the pipeline.

Vertex Shader: Programmable stage — transform each vertex from model to clip space.

Pipeline Stages at a Glance

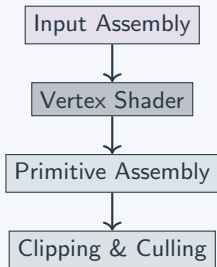


Input Assembly: Pull vertex data (positions, normals, UVs) into the pipeline.

Vertex Shader: Programmable stage — transform each vertex from model to clip space.

Primitive Assembly: Group vertices into triangles (or other primitives).

Pipeline Stages at a Glance



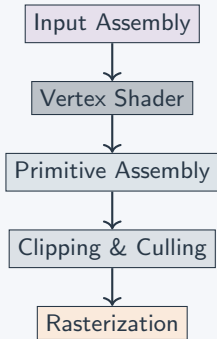
Input Assembly: Pull vertex data (positions, normals, UVs) into the pipeline.

Vertex Shader: Programmable stage — transform each vertex from model to clip space.

Primitive Assembly: Group vertices into triangles (or other primitives).

Clipping & Culling: Discard or trim primitives outside the view frustum.

Pipeline Stages at a Glance



Input Assembly: Pull vertex data (positions, normals, UVs) into the pipeline.

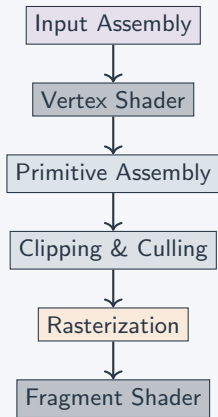
Vertex Shader: Programmable stage — transform each vertex from model to clip space.

Primitive Assembly: Group vertices into triangles (or other primitives).

Clipping & Culling: Discard or trim primitives outside the view frustum.

Rasterization: Convert triangles into a grid of fragments (potential pixels).

Pipeline Stages at a Glance



Input Assembly: Pull vertex data (positions, normals, UVs) into the pipeline.

Vertex Shader: Programmable stage — transform each vertex from model to clip space.

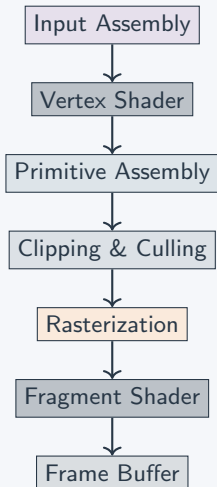
Primitive Assembly: Group vertices into triangles (or other primitives).

Clipping & Culling: Discard or trim primitives outside the view frustum.

Rasterization: Convert triangles into a grid of fragments (potential pixels).

Fragment Shader: Programmable stage — compute final color of each fragment.

Pipeline Stages at a Glance



Input Assembly: Pull vertex data (positions, normals, UVs) into the pipeline.

Vertex Shader: Programmable stage — transform each vertex from model to clip space.

Primitive Assembly: Group vertices into triangles (or other primitives).

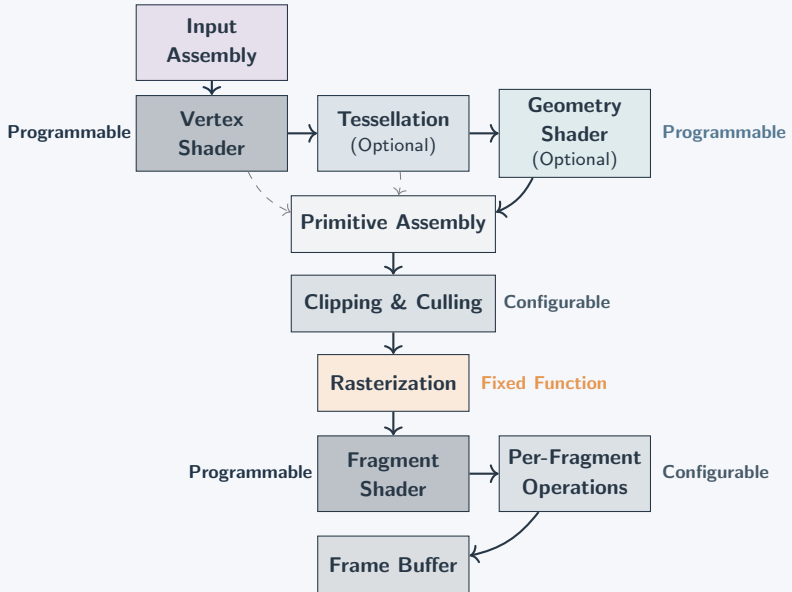
Clipping & Culling: Discard or trim primitives outside the view frustum.

Rasterization: Convert triangles into a grid of fragments (potential pixels).

Fragment Shader: Programmable stage — compute final color of each fragment.

Frame Buffer: Blend, depth-test, and write pixels to the screen.

Modern Advanced Pipeline



Data Flow Through the Pipeline

Input Data



3D Vertices

Data Flow Through the Pipeline

Input Data



3D Vertices

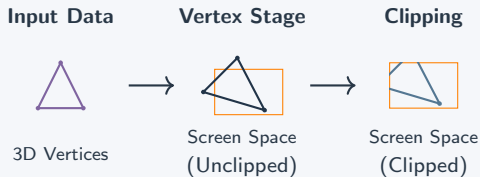


Vertex Stage

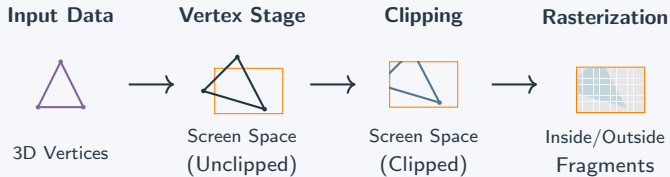


Screen Space
(Unclipped)

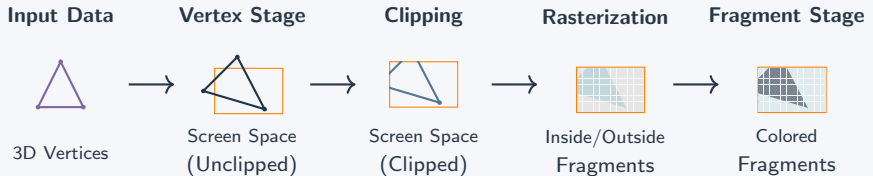
Data Flow Through the Pipeline



Data Flow Through the Pipeline



Data Flow Through the Pipeline



Programmable vs Fixed Function Stages

Programmable Stages

You write the code:

- **Vertex Shader:**
Transform positions,
compute lighting
- **Tessellation:** Subdivide
surfaces adaptively
- **Geometry Shader:**
Generate/modify
primitives
- **Fragment Shader:**
Compute final pixel colors

Maximum flexibility

Programmable vs Fixed Function Stages

Programmable Stages

You write the code:

- **Vertex Shader:**
Transform positions, compute lighting
- **Tessellation:** Subdivide surfaces adaptively
- **Geometry Shader:**
Generate/modify primitives
- **Fragment Shader:**
Compute final pixel colors

Maximum flexibility

Fixed Function Stages

Hardware handles it:

- **Primitive Assembly:**
Group vertices into triangles
- **Clipping:** Remove off-screen geometry
- **Rasterization:** Convert triangles to pixels
- **Depth Testing:** Z-buffer comparisons

Maximum performance

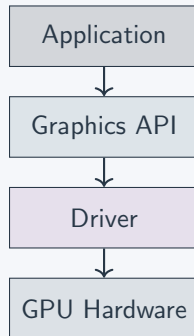
APIs & Shading Languages

Graphics APIs Overview

What is a Graphics API?

An **Application Programming Interface** that provides:

- Commands to control the GPU
- Abstraction over hardware differences
- Standard interface for graphics operations



OpenGL

Open Graphics Library

Perhaps the most widely used and most beginner-friendly graphics API. OpenGL was designed to be a cross-platform standard for rendering 2D and 3D graphics.

- Stable APIs
- High-level abstraction
- , ,  Support
- Two modes -
 - **Immediate Mode** - Deprecated, fixed function (Used by iGraphics)
 - **Retained Mode** - Modern OpenGL, uses shaders



Vulkan

Low-overhead, Cross-platform Graphics API

Developed by the Khronos Group as a modern successor to OpenGL. Designed for high-performance, multi-threaded rendering.

- Low-level control over GPU
- Better CPU-GPU parallelism
- Explicit memory and resource management
- , ,  Support
-  Support via MoltenVK



Major Graphics APIs

DirectX (Direct3D)

Microsoft's Graphics API for Windows and Xbox

A powerful API suite used primarily for game development on Windows platforms.

- Direct3D for 3D rendering
- Deep integration with Windows OS and drivers
- High performance with hardware vendor optimizations
-   Support only



Metal

Apple's Low-level Graphics API

Designed to maximize performance on Apple devices, replacing OpenGL on Apple platforms.

- Low-overhead, low-level access
- Unified graphics and compute
- Tight integration with Apple hardware
- 🍏 Support only (macOS, iOS, iPadOS)



Purpose

Shading languages allow programmers to write code that runs on the GPU for:

- **Vertex processing**
(transformations)
- **Fragment processing**
(lighting, texturing, effects)
- **Compute operations**
(general-purpose GPU computing)

Major Shading Languages:

- **GLSL** (OpenGL)
- **HLSL** (DirectX)
- **MSL** (Metal)
- **SPIR-V** (Vulkan)
Can be compiled from
GLSL or HLSL

GLSL: OpenGL Shading Language

GLSL Characteristics

- C-like syntax
- Built-in vector/matrix types
- Version-specific features

Data Types

- float, int, bool
- vec2, vec3, vec4
- mat2, mat3, mat4
- sampler2D, samplerCube

```
#version 330 core
// Vertex shader inputs
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aNormal;
layout (location = 2) in vec2 aTexCoord;
// Outputs to fragment shader
out vec3 FragPos;
out vec3 Normal;
out vec2 TexCoord;
// Uniform variables
uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;

void main() {
    FragPos = vec3(model * vec4(aPos, 1.0));
    Normal = mat3(transpose(inverse(model)))
               * aNormal;
    TexCoord = aTexCoord;

    gl_Position = projection
                   * view
                   * vec4(FragPos, 1.0);
}
```

Storage Qualifiers

- `in` - Input from previous stage
- `out` - Output to next stage
- `uniform` - Buffer from CPU

Layout Qualifiers

Used to explicitly assign indices or binding points to resources.

- `location` - Attribute/output index
- `binding` - Texture/uniform buffer slot

```
#version 330 core
// Fragment shader
in vec3 FragPos;
in vec3 Normal;
in vec2 TexCoord;
out vec4 FragColor;
uniform vec3 lightPos;
uniform vec3 viewPos;
uniform sampler2D texture_diffuse1;
void main() {
    vec3 color = texture(texture_diffuse1,
                          TexCoord).rgb;

    // Ambient
    vec3 ambient = 0.1 * color;
    // Diffuse
    vec3 norm = normalize(Normal);
    vec3 lightDir = normalize(
        lightPos - FragPos);
    float diff = max(dot(norm, lightDir),
                     0.0);
    vec3 diffuse = diff * color;
    // Result
    vec3 result = ambient + diffuse;
    FragColor = vec4(result, 1.0);
}
```


Input Assembly

Input Assembly Stage

Input Assembly

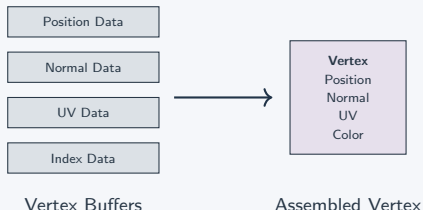
Input: Vertex data from application

Output: Organized vertex streams for vertex shader

Purpose:

- Pull vertex data from memory
- Organize data into vertex attributes
- Handle indexed vs non-indexed drawing
- Set up primitive topology

Configurable **fixed-function stage**



Vertex Attributes

Common Vertex Attributes

- **Position:** 3D coordinates (vec3)
- **Normal:** Surface normal vector (vec3)
- **Texture Coordinates:** UV mapping (vec2)
- **Color:** Vertex color (vec3/vec4)

Vertex Layout Example

$$\text{Vertex} = \begin{cases} \text{Position:} & (x, y, z) \\ \text{Normal:} & (n_x, n_y, n_z) \\ \text{UV:} & (u, v) \\ \text{Color:} & (r, g, b, a) \end{cases} \quad (1)$$

Total size: $3 + 3 + 2 + 4 = 12$ floats = 48 bytes

Position

Normal

UV

Color

12 bytes

12 bytes

8 bytes

16 bytes

Primitive Topology

Primitive Types

Points: Individual vertices

- Used for particle systems
- Point sprites

Lines: Connected line segments

- Wireframe rendering
- Debug visualization

Triangles: Most common primitive

- Standard for 3D surfaces
- Hardware optimized

Points



Lines



Triangles



Indexed vs Non-Indexed Drawing

Non-Indexed Drawing

Direct vertex specification

Each vertex is specified multiple times for shared vertices.

Problem:

- Vertex duplication
- Increased memory usage
- Inefficient for complex meshes

Indexed Drawing

Vertices referenced by indices

Each vertex is stored once, and indices specify how to connect them.

Benefits:

- No vertex duplication
- Lower memory usage
- Vertex cache friendly

Non-Indexed

Vertices: A, B, C, B, C, D

33% less data



Indexed

Vertices: A, B, C, D

Indices: 0, 1, 2, 1, 2, 3

Vertex Shader

Vertex Shader Stage

Vertex Shader

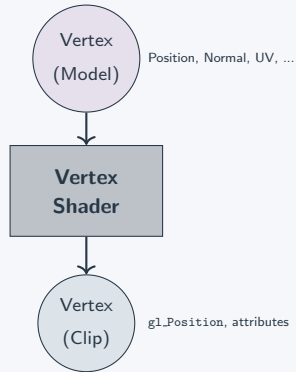
Input: Individual vertices with attributes

Output: Transformed vertices in clip space

Purpose:

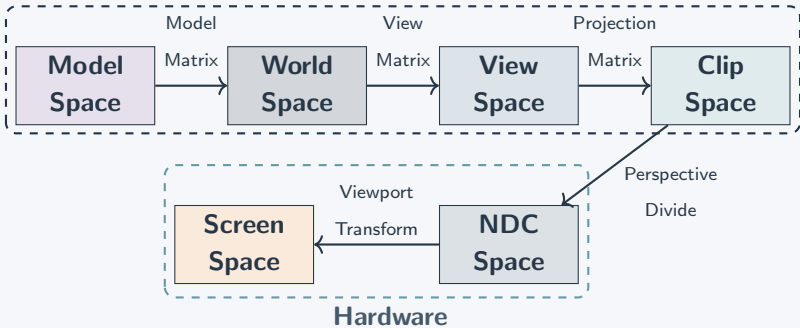
- Transform vertex positions through coordinate spaces
- Pass attributes to next stage
- Optional - Apply animations and deformations
- Optional - Calculate per-vertex lighting (Gouraud shading)

Programmable stage - you write the code!



The Transformation Pipeline

Vertex Shader



Transformation Matrices

Model Matrix

Purpose: Object-to-world transformation

$$\mathbf{M} = \mathbf{T} \cdot \mathbf{R} \cdot \mathbf{S}$$

- **T**: Translation
- **R**: Rotation
- **S**: Scale

Transforms from model's local coordinates to world coordinates.

View Matrix

Purpose: World-to-camera transformation

$$\mathbf{V} = \text{lookAt}(\text{eye}, \text{target}, \text{up})$$

Transforms from world coordinates to camera/eye coordinates.

Camera is at origin, looking down -Z axis.

Transformation Matrices

Projection Matrix

Purpose: Camera-to-clip space transformation

Perspective:

$$\mathbf{P}_{\text{persp}} = \begin{pmatrix} \frac{1}{\tan(\text{fov}/2) \cdot \text{aspect}} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\text{fov}/2)} & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Orthographic:

$$\mathbf{P}_{\text{ortho}} = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{-2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Clipping and Perspective Division

Clipping Phase

Purpose: Remove primitives outside the viewing frustum

- Operates in **clip space** (homogeneous coordinates)
- Keeps only geometry inside the cube: $-w \leq x, y, z \leq w$
- Output goes through **perspective division**:

$$\text{NDC} = \left(\frac{x}{w}, \frac{y}{w}, \frac{z}{w} \right)$$

Normalized Device Coordinates (NDC)

Canonical cube from $(-1, -1, -1)$ to $(1, 1, 1)$

- Independent of screen resolution
- Defines final visible region before pixel conversion
- x : left/right, y : bottom/top, z : near/far

Viewport Transformation and Screen Space

Viewport Transformation

Maps NDC coordinates to screen pixels:

$$\text{screen}_x = \left(\frac{\text{ndc}_x + 1}{2} \right) \cdot \text{width} + x_{\text{offset}}$$

$$\text{screen}_y = \left(\frac{\text{ndc}_y + 1}{2} \right) \cdot \text{height} + y_{\text{offset}}$$

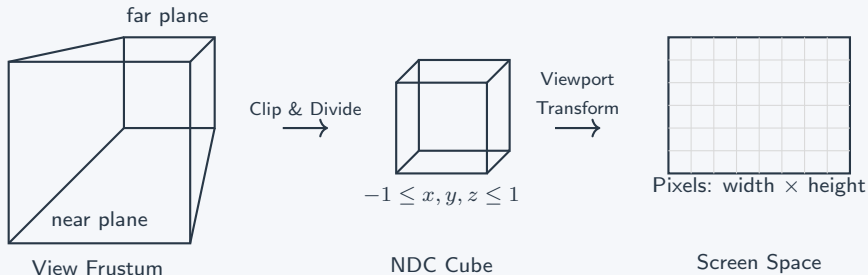
- Converts from $[-1, 1]$ range to pixel coordinates
- Accounts for viewport size and position

Screen Space

Final 2D position used for rasterization

- Measured in pixels
- Used by rasterizer to generate fragments

Frustum → NDC → Screen Space



Vertex Shader Example

```
#version 330 core
// Vertex shader inputs
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aNormal;
layout (location = 2) in vec2 aTexCoord;
// Outputs to fragment shader
out vec3 FragPos;
out vec3 Normal;
out vec2 TexCoord;
// Uniform variables
uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;

void main() {
    FragPos = vec3(model * vec4(aPos, 1.0));
    Normal = mat3(transpose(inverse(model)))
              * aNormal;
    TexCoord = aTexCoord;

    gl_Position = projection
                  * view
                  * vec4(FragPos, 1.0);
}
```

Output Variables

gl_Position -
Clip space position

Gouraud Shading

Gouraud (Per-Vertex) Shading

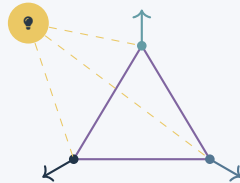
Compute lighting at vertices, interpolate across triangles

Process:

1. Calculate lighting at each vertex
2. Output vertex color
3. Hardware interpolates colors across triangle

Pros: Fast, good for distant objects

Cons: Poor specular highlights, faceted appearance



Flat



Gouraud



Phong

Tessellation Shader

Tessellation Shader Stage

Tessellation Shader

Input: Primitives

Output: Subdivided primitives with more vertices

Purpose:

- Subdivide low-poly meshes into high-poly
- Level-of-detail (LOD) based on distance
- Smooth curved surfaces
- Optional - Displacement mapping

Programmable stage - you write the code!

Input Patch

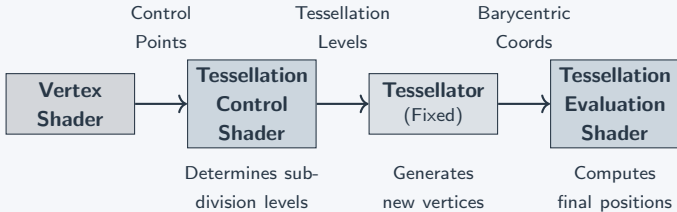


Tessellation



Tessellated

Tessellation Sub-Stages



Tessellation Use Cases

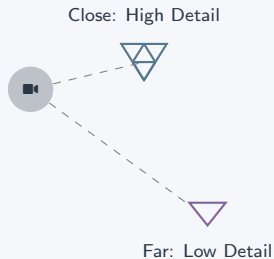
Level of Detail (LOD)

Adaptive subdivision based on:

- Distance from camera
- Screen space size
- Surface curvature
- Performance requirements

Benefits:

- Optimal vertex count
- Smooth transitions
- Better performance



Geometry Shader

Geometry Shader Stage

Geometry Shader

Input: Primitives (points, lines, triangles)

Output: New primitives (can generate or discard)

Purpose:

- Generate new geometry from existing primitives
- Discard primitives based on conditions
- Transform primitive types
- Add detail or effects

Programmable stage - you write the code!



Billboarding: Use Case

Billboarding

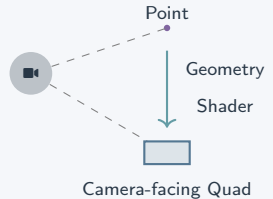
Convert points to camera-facing quads

Applications:

- Grass, leaves
- Particle systems
- Sprites and icons

Process:

1. **Input:** Single point
2. Calculate camera-facing orientation
3. **Output:** 4 vertices forming a quad



Primitive Assembly

Primitive Assembly Stage

Primitive Assembly

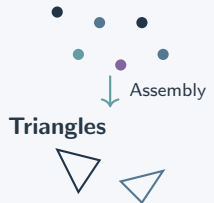
Input: Individual vertices from vertex/geometry shaders

Output: Complete primitives (triangles, lines, points)

Purpose:

- Group vertices into geometric primitives
- Establish winding order
- Prepare for clipping and culling

Individual Vertices



Winding Order & Face Orientation

Winding Order

Determines which side of triangle is "front"

Counter-Clockwise (CCW):

- OpenGL default
- Front-facing when viewed from front
- Vertices ordered: $A \rightarrow B \rightarrow C$

Clockwise (CW):

- DirectX default
- Front-facing when viewed from front
- Vertices ordered: $A \rightarrow C \rightarrow B$

Counter-Clockwise



Clockwise



Back-Face vs Front-Face

Face Determination

Process:

1. Calculate triangle normal using cross product
2. Determine viewing direction
3. Compare normal and view direction
4. Classify as front-face or back-face

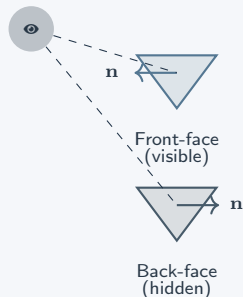
Mathematical test:

$$\mathbf{n} = (\mathbf{v}_1 - \mathbf{v}_0) \times (\mathbf{v}_2 - \mathbf{v}_0)$$

$$\text{facing} = \mathbf{n} \cdot \text{view_dir}$$

If facing $> 0 \rightarrow$ Front-face

If facing $< 0 \rightarrow$ Back-face



Clipping & Culling

Clipping & Culling Stage

Clipping & Culling

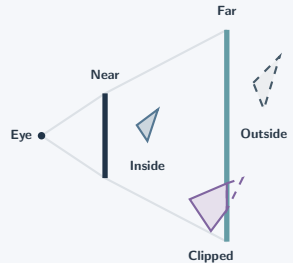
Input: Assembled primitives in clip space

Output: Visible primitives ready for rasterization

Purpose:

- Remove geometry outside the view frustum
- Eliminate back-facing triangles
- Reduce work for subsequent stages

Configurable fixed-function stage



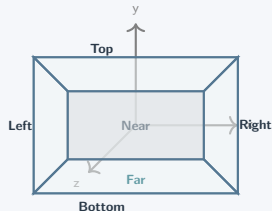
View Frustum & Clipping Planes

View Frustum Planes

Six clipping planes in clip space:

- **Near:** $z \geq -w$
- **Far:** $z \leq w$
- **Left:** $x \geq -w$
- **Right:** $x \leq w$
- **Bottom:** $y \geq -w$
- **Top:** $y \leq w$

After projection matrix transforms world coordinates to clip space (x, y, z, w) .



Clipping Test in Homogeneous Coordinates

For a vertex in clip space (x, y, z, w) , it's inside the frustum if:

$$-w \leq x \leq w \quad \text{and} \quad -w \leq y \leq w \quad \text{and} \quad -w \leq z \leq w$$

Clipping Methods: Homogeneous vs. Screen Space

Homogeneous Clipping

Stage: Before perspective divide (Clip Space)

Process:

1. Test vertices against six clip planes
2. Classify primitives (inside, outside, partial)
3. Compute intersection points if partially inside
4. Triangulate resulting polygon

Advantages:

- Linear equations:
 $ax + by + cz + dw = 0$
- Standard in GPU hardware

Typical Use: 3D scene rendering

Screen Space Clipping

Stage: After perspective divide (NDC)

Process:

1. Apply perspective divide
2. Use 2D clipping algorithms (e.g., Cohen-Sutherland)
3. Clip against screen-aligned rectangles

Characteristics:

- Simpler 2D math
- Often implemented per-pixel
- Good for scissor test, UI, and 2D overlays

Typical Use: 2D GUI rendering

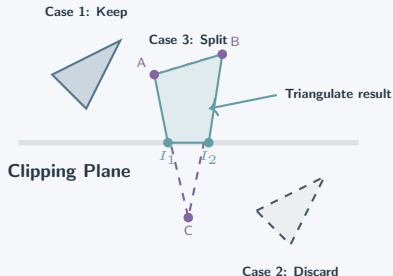
Note: Modern GPUs primarily perform homogeneous clipping in hardware, followed by optional screen-space techniques for specific tasks.

Triangle Clipping: Classification & Intersection

Triangle Classification

Vertex classification:

- **Fully Inside:** All vertices inside frustum $\Rightarrow$ Keep triangle
- **Fully Outside:** All vertices outside a single plane $\Rightarrow$ Discard
- **Intersecting:** Partially inside $\Rightarrow$ Clip and generate polygon



Triangle Clipping: Classification & Intersection

Edge-Plane Intersection

For a clipping plane $ax+by+cz+dw = 0$, and an edge $(\mathbf{P}_1, \mathbf{P}_2)$:

$$d_i = ax_i + by_i + cz_i + dw_i$$

$$t = \frac{d_1}{d_1 - d_2}$$

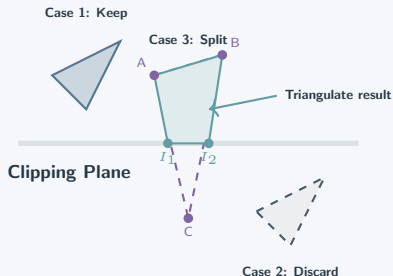
$$\mathbf{P}_{\text{int}} = \mathbf{P}_1 + t(\mathbf{P}_2 - \mathbf{P}_1)$$

New intersection points lie on the clipping boundary.

Polygon Reconstruction

Clipping may produce convex polygons with 4+ vertices.

These must be **triangulated** (e.g., using a fan from one vertex) before rasterization.



Sutherland-Hodgman Clipping Algorithm

Sequential Plane Clipping

Process each plane in sequence

For each clipping plane:

1. Start with current polygon
2. Process each edge against plane
3. Build output vertex list
4. Use output as input for next plane

Edge processing rules:

- Inside $\rightarrow$ Inside: Output end vertex
- Inside $\rightarrow$ Outside: Output intersection
- Outside $\rightarrow$ Inside: Output intersection + end
- Outside $\rightarrow$ Outside: Output nothing

Step 1: Original Triangle



Input: 3 vertices

Step 2: Left Plane



Output: 4 vertices

Step 3: Right Plane



Output: 5 vertices

Step 4: All Planes



Final clipped polygon

Back-Face Culling

Back-Face Culling

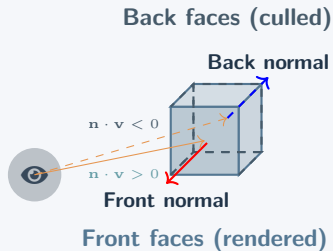
Purpose: Remove triangles facing away from viewer

Method: Test triangle normal against view direction

Cull if: $\mathbf{n} \cdot \mathbf{v} < 0$

Benefits:

- 50% triangle reduction (closed meshes)
- Reduced overdraw
- Less fragment shading



Rasterization

Rasterization Stage

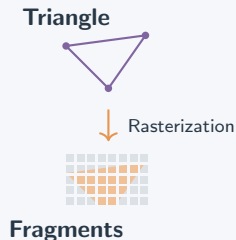
Rasterization

Input: Clipped triangles in screen space

Output: Fragments with interpolated attributes

Purpose:

- Convert vector triangles to discrete fragments
- Determine which pixels are covered
- Interpolate vertex attributes across triangles
- Generate depth values for visibility testing



Triangle Rasterization Algorithm

Rasterization Steps

1. Bounding Box Calculation

- Find min/max x,y coordinates
- Limit to screen boundaries

2. Fragment Generation

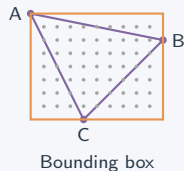
- Test each pixel in bounding box
- Determine if pixel center is inside triangle

3. Attribute Interpolation

- Calculate barycentric coordinates
- Interpolate vertex attributes

4. Depth Calculation

- Interpolate z-values
- Prepare for depth testing



Edge Functions & Half-Space Test

Edge Function Method

For triangle with vertices A, B, C:

Define edge functions:

$$E_{AB}(x, y) = (B_x - A_x)(y - A_y) - (B_y - A_y)(x - A_x)$$

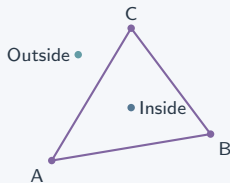
$$E_{BC}(x, y) = (C_x - B_x)(y - B_y) - (C_y - B_y)(x - B_x)$$

$$E_{CA}(x, y) = (A_x - C_x)(y - C_y) - (A_y - C_y)(x - C_x)$$

Inside test: Point (x, y) is inside if:

$$E_{AB}(x, y) \geq 0 \text{ and } E_{BC}(x, y) \geq 0 \text{ and } E_{CA}(x, y) \geq 0$$

(Assuming counter-clockwise winding)



Barycentric Coordinates

Barycentric Interpolation

Express any point **P** inside triangle as:

$$\mathbf{P} = \alpha \mathbf{A} + \beta \mathbf{B} + \gamma \mathbf{C}$$

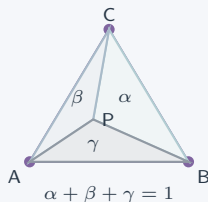
where $\alpha + \beta + \gamma = 1$ and $\alpha, \beta, \gamma \geq 0$

Calculation:

$$\alpha = \frac{E_{BC}(\mathbf{P})}{E_{BC}(\mathbf{A})}$$

$$\beta = \frac{E_{CA}(\mathbf{P})}{E_{CA}(\mathbf{B})}$$

$$\gamma = \frac{E_{AB}(\mathbf{P})}{E_{AB}(\mathbf{C})}$$



Fragment Attribute Interpolation

Interpolated Attributes

Using barycentric coordinates:

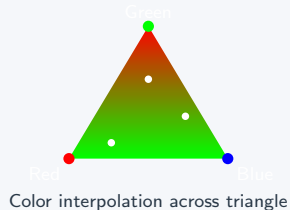
For any vertex attribute **attr**:

$$\mathbf{attr}_P = \alpha \mathbf{attr}_A + \beta \mathbf{attr}_B + \gamma \mathbf{attr}_C$$

Common interpolated attributes:

- Texture coordinates (UV)
- Vertex colors
- Normal vectors
- World positions
- Custom vertex data

Note: Hardware performs perspective-correct interpolation



Z-Buffer Value Interpolation

Depth Value Generation

Linear interpolation in screen space:

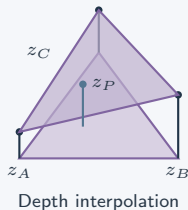
$$z_P = \alpha z_A + \beta z_B + \gamma z_C$$

where z_A, z_B, z_C are the depth values of triangle vertices.

Depth range mapping:

- NDC depth: $[-1, 1]$ (OpenGL) or $[0, 1]$ (DirectX)
- Higher precision near near plane

Usage: Depth testing, early Z rejection



Fragment Properties

Each generated fragment contains:

- **Screen coordinates** (x , y)
- **Depth value** (z)
- **Interpolated attributes** (colors, UVs, normals, etc.)
- **Coverage information** (for anti-aliasing)

Fragment $\neq$ Pixel:

- Fragments are candidates for pixels
- Multiple fragments may affect one pixel
- Some fragments may be discarded

Fragment Shader

Fragment Shader Stage

Fragment Shader

Input: Fragments with interpolated attributes

Output: Final fragment colors (and depth)

Primary responsibility:

- Compute final color for each fragment



Fragments

Fragment Shader Stage

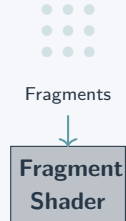
Fragment Shader

Input: Fragments with interpolated attributes

Output: Final fragment colors (and depth)

Primary responsibility:

- Compute final color for each fragment
- Apply lighting calculations



Fragment Shader Stage

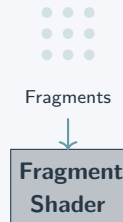
Fragment Shader

Input: Fragments with interpolated attributes

Output: Final fragment colors (and depth)

Primary responsibility:

- Compute final color for each fragment
- Apply lighting calculations
- Sample textures



Fragment Shader Stage

Fragment Shader

Input: Fragments with interpolated attributes

Output: Final fragment colors (and depth)

Primary responsibility:

- Compute final color for each fragment
- Apply lighting calculations
- Sample textures
- Implement material properties



Fragment Shader Stage

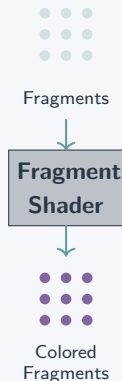
Fragment Shader

Input: Fragments with interpolated attributes

Output: Final fragment colors (and depth)

Primary responsibility:

- Compute final color for each fragment
- Apply lighting calculations
- Sample textures
- Implement material properties
- Output color and optional data

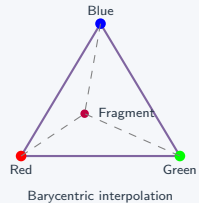


Fragment Shader Inputs

Interpolated Attributes

From vertex shader via rasterization:

- **Position** - World space coordinates
- **Normal vectors** - For lighting calculations
- **Texture coordinates** - UV mapping
- **Color** - Vertex colors
- **Custom attributes** - Application-specific data

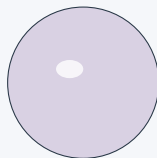


Per-Fragment Lighting

Phong Shading

Phong Shading (Per-Fragment):

- Interpolate normals, not colors
- Compute lighting at each fragment
- Higher quality, smooth specular highlights
- More computationally expensive

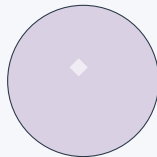


Phong Shading

Gouraud Shading

Gouraud Shading (Per-Vertex):

- Compute lighting at vertices
- Interpolate final colors
- Faster but lower quality
- Poor specular highlights



Gouraud Shading

Fragment Shader - Lighting Example

```
#version 330 core
// Fragment shader inputs
in vec3 FragPos;    // World position
in vec3 Normal;     // Interpolated normal
in vec2 TexCoord;   // Texture coordinates
// Outputs
out vec4 FragColor;
// Uniforms (lighting parameters)
uniform vec3 lightPos;
uniform vec3 lightColor;
uniform vec3 viewPos;
uniform sampler2D diffuseTexture;
void main() {
    // Sample texture
    vec3 albedo = texture(diffuseTexture,
                          TexCoord).rgb;

    // Normalize interpolated normal
    vec3 norm = normalize(Normal);
    // Light direction
    vec3 lightDir = normalize(lightPos
                              - FragPos);

    // Ambient
    vec3 ambient = 0.15 * albedo;
    // Diffuse (Lambert)
    float diff = max(dot(norm, lightDir),
                     0.0);

    vec3 diffuse = diff * lightColor
                  * albedo;

    // Specular (Blinn-Phong)
    vec3 viewDir = normalize(viewPos
                              - FragPos);
    vec3 halfwayDir = normalize(
        lightDir + viewDir);
    float spec = pow(max(
        dot(norm, halfwayDir),
        0.0), 64.0);
    vec3 specular = spec * lightColor;
    // Combine lighting components
    vec3 result = ambient + diffuse
                + specular;
    FragColor = vec4(result, 1.0);
}
```

Multiple Texture Sampling Example

```
// Multiple texture samplers
uniform sampler2D diffuseMap;
uniform sampler2D normalMap;
uniform sampler2D specularMap;
uniform sampler2D roughnessMap;

void main() {
    // Sample multiple textures
    vec3 albedo = texture(diffuseMap,
                          TexCoord).rgb;
    vec3 normal = texture(normalMap,
                          TexCoord).rgb;
    float specular = texture(specularMap,
                          TexCoord).r;
    float roughness = texture(roughnessMap,
                          TexCoord).r;

    // Convert normal map from [0,1] to [-1,1]
    normal = normal * 2.0 - 1.0;

    // Transform normal to world space
    // (requires TBN matrix)

    // Use in lighting calculations...
}
```

Fragment Shader Outputs

Standard Outputs

Primary output: RGBA

RGBA Color:

- Red, Green, Blue channels (0.0 to 1.0)
- Alpha channel for transparency

Depth modification:

- Can output custom depth values
- Used for special effects
- Can break early depth testing

Fragment Shader - Programmable Stage

Highly Programmable

Most flexible stage in the graphics pipeline

Programmable aspects:

- Custom lighting models (Phong, Blinn-Phong, PBR, etc.)
- Complex material properties and texturing
- Advanced effects (normal mapping, parallax mapping)
- Post-processing effects
- Procedural generation

Fragment Shader - Programmable Stage

Highly Programmable

Most flexible stage in the graphics pipeline

Programmable aspects:

- Custom lighting models (Phong, Blinn-Phong, PBR, etc.)
- Complex material properties and texturing
- Advanced effects (normal mapping, parallax mapping)
- Post-processing effects
- Procedural generation

Performance Considerations

Most computationally expensive stage

- Executes once per visible fragment
- Can be millions of invocations per frame

Per-Fragment Operations

Per-Fragment Operations Stage

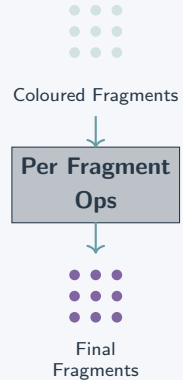
Per-Fragment Operations

Input: Colored fragments from fragment shader

Output: Final pixels to framebuffer

Fixed-function tests and operations:

- Scissor test
- Stencil test
- Depth test
- Alpha blending

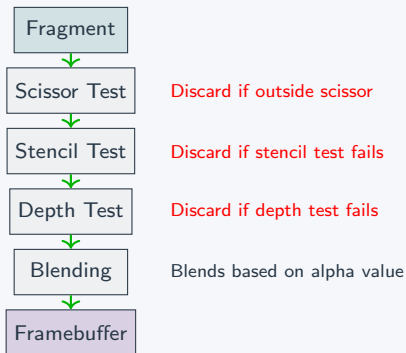


Per-Fragment Operations Pipeline

Fixed-Function Stage

Non-programmable stage with configurable tests and operations

Sequential processing: Each fragment goes through all enabled tests



Depth Testing

Z-Buffer Algorithm

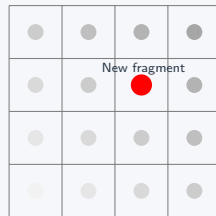
Purpose: Determine which fragments are visible

Process:

1. Compare fragment depth with stored depth
2. If test passes, update color and depth buffers
3. If test fails, discard fragment

Common depth functions:

- `GL_LESS` - Default, closer objects win
- `GL_LEQUAL` - Less than or equal
- `GL_GREATER` - Farther objects win
- `GL_ALWAYS` - Always pass (no depth testing)



Depth Buffer

If $z_{new} < z_{stored}$, replace

Early Z-Testing Optimization

Early Depth Test

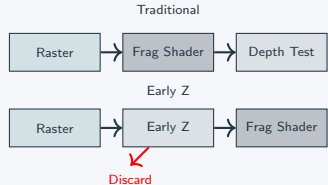
Optimization: Perform depth test *before* fragment shader

Benefits:

- Avoid expensive fragment shader execution
- Significant performance improvement
- Especially effective for complex shaders

Limitations:

- Fragment shader cannot modify depth
- No alpha testing in fragment shader
- Must be enabled/supported by hardware



Alpha Blending

Transparency via Blending

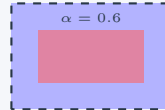
Purpose: Combine fragment color with framebuffer color

Standard alpha blending:

$$C_{result} = C_{src} \cdot \alpha_{src} + C_{dst} \cdot (1 - \alpha_{src})$$

Blend equation components:

- C_{src} = source color (fragment)
- C_{dst} = destination color (framebuffer)
- α_{src} = source alpha value



Background

Blend Functions

Common source factors:

- GL_SRC_ALPHA
- GL_ONE
- GL_ZERO

Common dest factors:

- GL_ONE_MINUS_SRC_ALPHA
- GL_ONE
- GL_DST_ALPHA

Advanced Blending Modes

Additive Blending

Formula: $C_{result} = C_{src} + C_{dst}$

Use cases:

- Fire, explosions, energy effects
- Light accumulation
- Particle systems

Multiplicative Blending

Formula: $C_{result} = C_{src} \times C_{dst}$

Use cases:

- Shadows, darkening effects
- Color filtering
- Atmospheric effects

Screen Blending

Formula: $C_{result} = 1 - (1 - C_{src})(1 - C_{dst})$

Use cases:

- Brightening effects
- Light overlays
- Highlight enhancement

Order Dependency

Problem: Blending is order-dependent

Solutions:

- Sort transparent objects back-to-front
- Order-independent transparency (OIT)

Additional Per-Fragment Operations

Stencil Test

Purpose: Per-pixel masking and control flow

How it works:

- Each pixel stores an 8-bit stencil value
- Compare fragment's stencil to buffer value
- Conditionally discard fragment based on test
- Can also update stencil buffer (e.g., increment)

Applications:

- Object outlining
- Mirror/portal rendering
- Shadow volumes
- UI masking

Scissor Test

Purpose: Rectangular clipping region

Use cases:

- Discard fragments outside defined rectangle
- UI rendering, viewport masking
- Very efficient and hardware-accelerated

Others

Less common operations:

- **Dithering:** Adds noise to smooth color gradients
- **Logic Ops:** Bitwise operations on output color
- **Occlusion Queries:** Count visible fragments

Framebuffer Output

Framebuffer Output Stage

Framebuffer Output

Input: Final fragment colors and depth values

Output: Pixels written to display

Purpose:

- Write final pixel data to memory
- Manage color format conversions
- Support double buffering for smooth display

Fixed-function stage

Fragment Data

Color (RGBA)

Depth (Z)



Framebuffer

Color Buffer
Depth Buffer
Stencil Buffer



Display

Framebuffer Structure

Buffer Types

Color Buffer:

- Stores final pixel colors
- Multiple render targets (MRT) support
- Various formats: RGBA8, RGBA16F, etc.

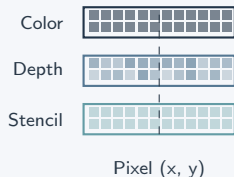
Depth Buffer:

- Stores depth values for Z-testing
- Typically 16, 24, or 32-bit precision
- Essential for 3D rendering

Stencil Buffer:

- 8-bit per-pixel values
- Used for advanced effects

Framebuffer Layout



Double Buffering

Double Buffering Concept

Problem without double buffering:

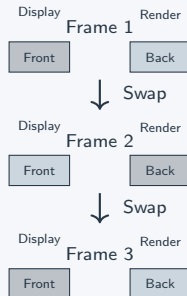
- Rendering directly to display buffer
- Visible tearing and flickering
- Partial frame updates visible to user

Double buffering solution:

1. Render to back buffer (off-screen)
2. When frame complete, swap buffers
3. Front buffer displayed while back buffer rendered
4. Atomic swap prevents tearing

Result: Smooth, tear-free animation

Buffer Swapping



VSync & Refresh Rates

Vertical Synchronization

VSync purpose:

- Synchronize buffer swaps with display refresh
- Prevent screen tearing
- Limit frame rate to display refresh rate

Common refresh rates:

- 60 Hz (16.67 ms per frame)
- 120 Hz (8.33 ms per frame)
- 144 Hz (6.94 ms per frame)

VSync Trade-offs

With VSync enabled:

- **Pro:** No tearing
- **Pro:** Consistent frame timing
- **Con:** Input latency increase
- **Con:** Frame rate capped

Without VSync:

- **Pro:** Lower input latency
- **Pro:** Uncapped frame rate
- **Con:** Screen tearing
- **Con:** Inconsistent timing

VSync Timing

Buffer Swaps



Framebuffer Summary

Key Concepts

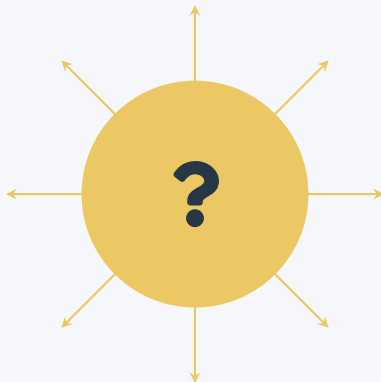
Framebuffer is the final destination:

- Collection of buffers (color, depth, stencil)
- Can render to screen or off-screen targets
- Managed through graphics API

Double buffering essential for:

- Smooth animation
- Preventing visual artifacts
- Professional presentation quality

Questions?



References & Further Reading

-  Peter Shirley and Steve Marschner et al. *Fundamentals of Computer Graphics (4th Edition)*. CRC Press, 2016.
Available as PDF
-  Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically Based Rendering: From Theory to Implementation (4th Edition)*. Morgan Kaufmann, 2023.
Available online
-  Peter Shirley. *Ray Tracing in One Weekend*. Self-published, 2016–2020.
Project Website
-  MIT OpenCourseWare: 6.837 Computer Graphics.
ocw.mit.edu/6-837
-  Scratchapixel: Learn Computer Graphics Programming.
scratchapixel.com